[inserir aqui a logo da universidade]

UNIVERSIDADE ESTÁCIO DE SÁ

Campus: Polo Alexandrino

Curso: Desenvolvimento Full Stack

Disciplina: DGT2811 – Desenvolvimento Back-End Corporativo com Java e Cloud

Turma: [preencher]

Semestre letivo: [preencher]

Integrante: Bárbara Floripes

Trabalho Prático

CadastroEE – Sistema cadastral Web com Servlets, JPA e JEE

Repositório: https://github.com/Barbysfn/Trabalho-Pratico-DGT2811-Desenvolvimento-Back-End-Corporativo-com-Java-e-Cloud

Sumário

Título da Prática..... 3
Objetivo da Prática..... 3
Ambiente utilizado..... 3
1° Procedimento – Camadas de Persistência e Controle..... 4
1.1 Conexão com o SQL Server e pool de conexões no GlassFish..... 4
1.2 Aplicativo corporativo CadastroEE..... 5
1.3 Ajustes para o Jakarta..... 5
1.4 Códigos – CadastroEE-ejb..... 6
1.5 Códigos – CadastroEE-war (Servlet de teste)..... 22
1.6 Execução..... 24
1.7 Análise e Conclusão..... 25
2° Procedimento – Interface Cadastral com Servlet e JSPs..... 27
2.1 Códigos..... 27
2.2 Resultados dos testes..... 31
2.3 Análise e Conclusão..... 33
3° Procedimento – Melhorando o Design da Interface..... 34
3.1 Códigos..... 34
3.2 Resultados..... 36
3.3 Análise e Conclusão..... 37
Conclusão geral..... 38

Título da Prática

Implementação de sistema cadastral com interface Web, baseado nas tecnologias de Servlets, JPA e JEE (projeto CadastroEE).

Objetivo da Prática

- Implementar persistência com base em JPA.
- Implementar regras de negócio na plataforma JEE, através de EJBs.
- Implementar sistema cadastral Web com base em Servlets e JSPs.
- Utilizar a biblioteca Bootstrap para melhoria do design.

Ao final, a aplicação permite listar, incluir, alterar e excluir produtos do banco loja (SQL Server) por meio de uma interface Web, passando pelas camadas Web (Servlets e JSPs), de controle (Session Beans) e de persistência (entidades JPA).

Ambiente utilizado

- Windows 11, JDK 17 (Eclipse Temurin 17.0.20) e Apache NetBeans IDE 25 (projeto Ant – Java Enterprise › Enterprise Application).
- GlassFish Server 6.2.5 (Jakarta EE 9.1 – por isso os pacotes javax foram trocados por jakarta).
- Microsoft SQL Server 2022 Express, instância padrão na porta 1433, banco loja com o login loja/loja.
- Driver JDBC mssql-jdbc-12.2.0.jre8.jar, copiado para glassfish6/glassfish/lib.
- Bootstrap 5.3.3 via CDN (jsDelivr).

Repositório Git com todo o código, scripts e este relatório: https://github.com/Barbysfn/Trabalho-Pratico-DGT2811-Desenvolvimento-Back-End-Corporativo-com-Java-e-Cloud.

1º Procedimento – Camadas de Persistência e Controle

1.1 Conexão com o SQL Server e pool de conexões no GlassFish

O driver do SQL Server foi registrado no NetBeans (Serviços › Banco de Dados › Drivers) e foi criada a conexão loja com a URL jdbc:sqlserver://localhost:1433;databaseName=loja;encrypt=true;trustServerCertificate=true;. O arquivo mssql-jdbc-12.2.0.jre8.jar foi copiado para o diretório lib do GlassFish e, com o servidor iniciado, foram executados no asadmin os comandos abaixo, exatamente como no roteiro (o identificador do pool, SQLServerPool, foi informado ao final do comando):

Comandos asadmin

```
create-jdbc-connection-pool
  --datasourceclassname com.microsoft.sqlserver.jdbc.SQLServerDataSource
  --restype javax.sql.DataSource
  --property
driverClass=com.microsoft.sqlserver.jdbc.SQLServerDriver:portNumber=1433:password=loja:user=loja:ser
verName=localhost:databaseName=loja:trustServerCertificate=true:URL="jdbc\\:sqlserver\\://
localhost\\:1433\\;databaseName\\=loja\\;encrypt\\=true\\;trustServerCertificate\\=true\\;"
  SQLServerPool

ping-connection-pool SQLServerPool

create-jdbc-resource --connectionpoolid SQLServerPool jdbc/loja
```

Resultado da execução

```
JDBC connection pool SQLServerPool created successfully.
Command create-jdbc-connection-pool executed successfully.
Command ping-connection-pool executed successfully.
JDBC resource jdbc/loja created successfully.
Command create-jdbc-resource executed successfully.
```

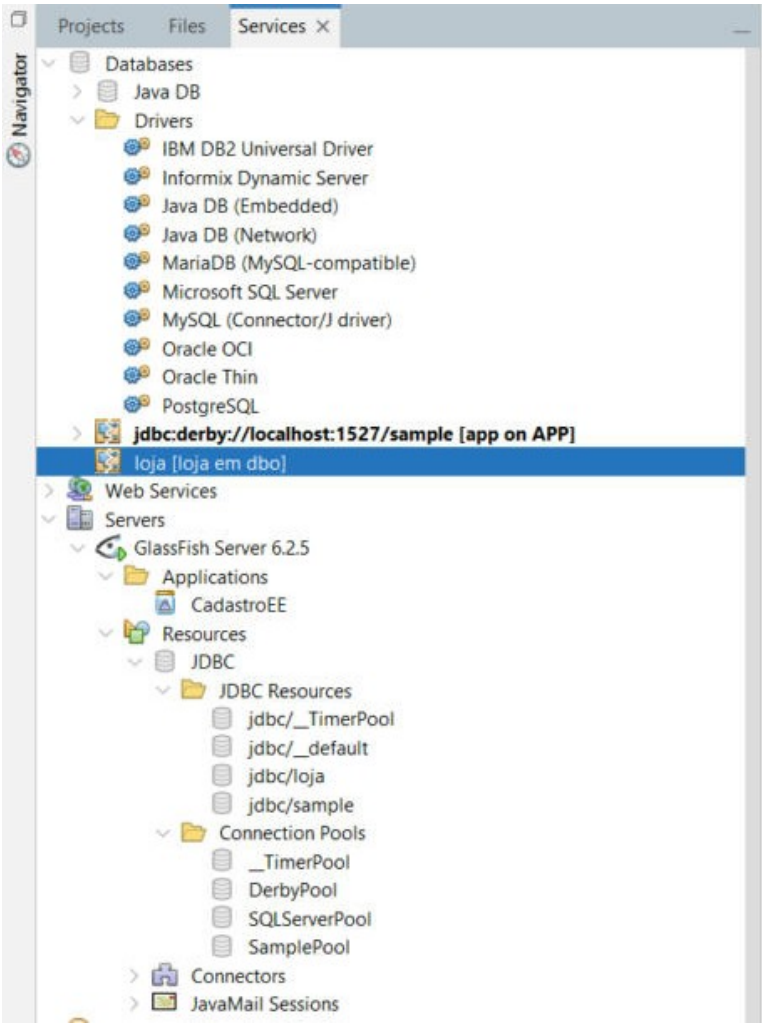


Figura 1 – Aba Serviços do NetBeans: driver Microsoft SQL Server, conexão loja, GlassFish 6.2.5 com o recurso jdbc/loja, o pool SQLServerPool e a aplicação CadastroEE implantada

1.2 Aplicativo corporativo CadastroEE

Foi criado o projeto Ant › Java Enterprise › Enterprise Application CadastroEE, com o servidor GlassFish e a plataforma Jakarta EE 8, que gerou os projetos dependentes CadastroEE-ejb (JPA e EJB) e CadastroEE-war (Web). No CadastroEE-ejb foram geradas as entidades com New Entity Classes from Database (Data Source jdbc/loja, todas as tabelas, pacote cadastroee.model e persistence.xml) e, em seguida, os Session Beans com New Session Beans for Entity Classes (todas as entidades, interface local, pacote cadastroee.controller — o roteiro traz o erro de digitação "cadatroee").

1.3 Ajustes para o Jakarta

- Biblioteca Jakarta EE 8 API adicionada ao CadastroEE-ejb e Jakarta EE Web 8 API ao CadastroEE-war.
- Todas as importações javax.* trocadas por jakarta.* nos dois projetos.
- Produto.precoVenda alterado de BigDecimal para Float.
- persistence.xml substituído pelo do roteiro e web.xml do CadastroEE-war substituído pelo do roteiro.
- Ajuste adicional: @GeneratedValue(strategy = GenerationType.IDENTITY) nas entidades cujas chaves são IDENTITY no SQL Server (Produto, Usuario, Movimento), necessário para a inclusão de novos produtos.

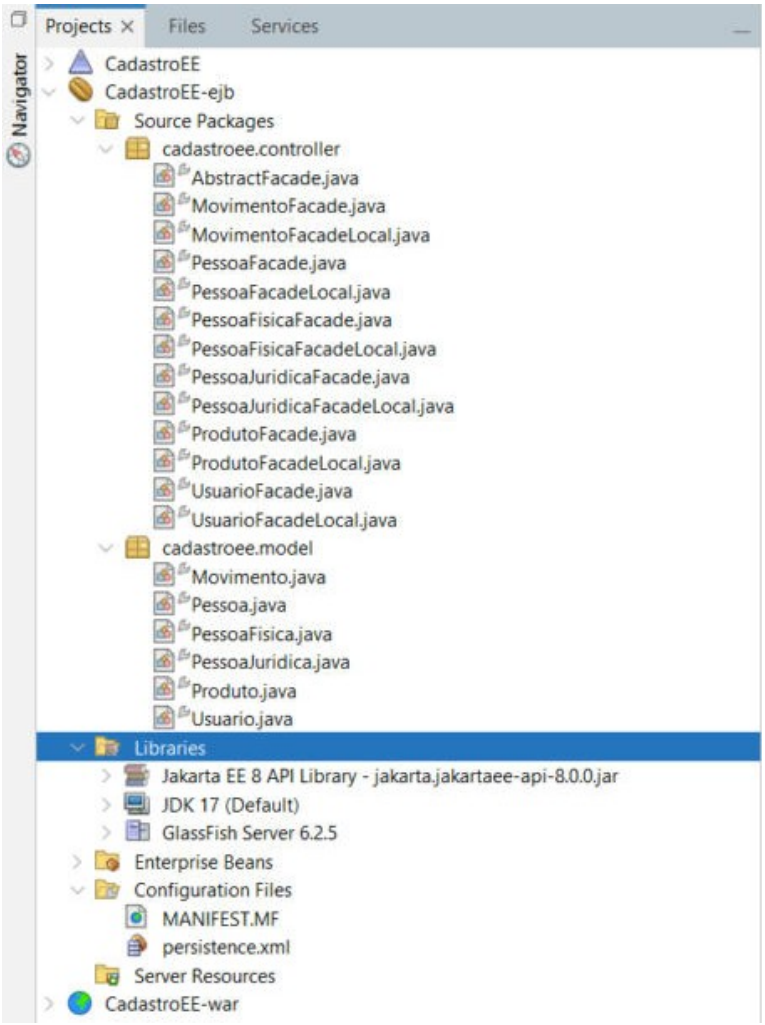


Figura 2 – Projeto CadastroEE-ejb no NetBeans: entidades, Session Beans, bibliotecas e persistence.xml

1.4 Códigos – CadastroEE-ejb

CadastroEE-ejb/src/conf/persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0"
xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
  <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties/>
  </persistence-unit>
</persistence>
```

CadastroEE-ejb/src/java/cadastroee/model/Produto.java

```
package cadastroee.model;

import java.io.Serializable;
import java.util.Collection;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

/**
 * Entidade JPA para a tabela Produto.
 * Gerada via "New Entity Classes from Database" e ajustada:
 *  - imports javax -> jakarta;
 *  - precoVenda de BigDecimal para Float;
 *  - @GeneratedValue(IDENTITY), pois idProduto é IDENTITY no SQL Server;
 *  - sem cascade em movimentoCollection, para que excluir um produto
 *    nunca apague o histórico de movimentos.
 */
@Entity
@Table(name = "Produto")
@NamedQueries({
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p"),
    @NamedQuery(name = "Produto.findByIdProduto", query = "SELECT p FROM Produto p WHERE p.idProduto
= :idProduto"),
    @NamedQuery(name = "Produto.findByNome", query = "SELECT p FROM Produto p WHERE p.nome
= :nome"),
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM Produto p WHERE
p.quantidade = :quantidade"),
    @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p FROM Produto p WHERE
p.precoVenda = :precoVenda")})
public class Produto implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "idProduto")
    private Integer idProduto;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 255)
    @Column(name = "nome")
    private String nome;
    @Basic(optional = false)
    @NotNull
    @Column(name = "quantidade")
    private int quantidade;
    @Basic(optional = false)
    @NotNull
    @Column(name = "precoVenda")
    private Float precoVenda;
    @OneToMany(mappedBy = "idProduto")
    private Collection<Movimento> movimentoCollection;

    public Produto() {
    }

    public Produto(Integer idProduto) {
        this.idProduto = idProduto;
    }
```

```
public Produto(Integer idProduto, String nome, int quantidade, Float precoVenda) {
    this.idProduto = idProduto;
    this.nome = nome;
    this.quantidade = quantidade;
    this.precoVenda = precoVenda;
}

public Integer getIdProduto() {
    return idProduto;
}

public void setIdProduto(Integer idProduto) {
    this.idProduto = idProduto;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getQuantidade() {
    return quantidade;
}

public void setQuantidade(int quantidade) {
    this.quantidade = quantidade;
}

public Float getPrecoVenda() {
    return precoVenda;
}

public void setPrecoVenda(Float precoVenda) {
    this.precoVenda = precoVenda;
}

public Collection<Movimento> getMovimentoCollection() {
    return movimentoCollection;
}

public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
    this.movimentoCollection = movimentoCollection;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idProduto != null ? idProduto.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    if (!(object instanceof Produto)) {
        return false;
    }
```

```
        Produto other = (Produto) object;
        return !((this.idProduto == null && other.idProduto != null)
                || (this.idProduto != null && !this.idProduto.equals(other.idProduto)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.Produto[ idProduto=" + idProduto + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/model/Pessoa.java

```
package cadastroee.model;

import java.io.Serializable;
import java.util.Collection;
import jakarta.persistence.Basic;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

/**
 * Entidade JPA para a tabela Pessoa (gerada a partir do banco "loja").
 */
@Entity
@Table(name = "Pessoa")
@NamedQueries({
    @NamedQuery(name = "Pessoa.findAll", query = "SELECT p FROM Pessoa p"),
    @NamedQuery(name = "Pessoa.findByIdPessoa", query = "SELECT p FROM Pessoa p WHERE p.idPessoa
= :idPessoa"),
    @NamedQuery(name = "Pessoa.findByNome", query = "SELECT p FROM Pessoa p WHERE p.nome = :nome"),
    @NamedQuery(name = "Pessoa.findByLogradouro", query = "SELECT p FROM Pessoa p WHERE p.logradouro
= :logradouro"),
    @NamedQuery(name = "Pessoa.findByCidade", query = "SELECT p FROM Pessoa p WHERE p.cidade
= :cidade"),
    @NamedQuery(name = "Pessoa.findByEstado", query = "SELECT p FROM Pessoa p WHERE p.estado
= :estado"),
    @NamedQuery(name = "Pessoa.findByTelefone", query = "SELECT p FROM Pessoa p WHERE p.telefone
= :telefone"),
    @NamedQuery(name = "Pessoa.findByEmail", query = "SELECT p FROM Pessoa p WHERE p.email
= :email")})
public class Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @Basic(optional = false)
    @NotNull
    @Column(name = "idPessoa")
    private Integer idPessoa;
    @Basic(optional = false)
```

```
    @NotNull
    @Size(min = 1, max = 255)
    @Column(name = "nome")
    private String nome;
    @Size(max = 255)
    @Column(name = "logradouro")
    private String logradouro;
    @Size(max = 255)
    @Column(name = "cidade")
    private String cidade;
    @Size(max = 2)
    @Column(name = "estado")
    private String estado;
    @Size(max = 11)
    @Column(name = "telefone")
    private String telefone;
    @Size(max = 255)
    @Column(name = "email")
    private String email;
    @OneToOne(cascade = CascadeType.ALL, mappedBy = "pessoa")
    private PessoaFisica pessoaFisica;
    @OneToOne(cascade = CascadeType.ALL, mappedBy = "pessoa")
    private PessoaJuridica pessoaJuridica;
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "idPessoa")
    private Collection<Movimento> movimentoCollection;

    public Pessoa() {
    }

    public Pessoa(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }

    public Integer getIdPessoa() {
        return idPessoa;
    }

    public void setIdPessoa(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getLogradouro() {
        return logradouro;
    }

    public void setLogradouro(String logradouro) {
        this.logradouro = logradouro;
    }

    public String getCidade() {
        return cidade;
    }
```

```
public void setCidade(String cidade) {
    this.cidade = cidade;
}

public String getEstado() {
    return estado;
}

public void setEstado(String estado) {
    this.estado = estado;
}

public String getTelefone() {
    return telefone;
}

public void setTelefone(String telefone) {
    this.telefone = telefone;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public PessoaFisica getPessoaFisica() {
    return pessoaFisica;
}

public void setPessoaFisica(PessoaFisica pessoaFisica) {
    this.pessoaFisica = pessoaFisica;
}

public PessoaJuridica getPessoaJuridica() {
    return pessoaJuridica;
}

public void setPessoaJuridica(PessoaJuridica pessoaJuridica) {
    this.pessoaJuridica = pessoaJuridica;
}

public Collection<Movimento> getMovimentoCollection() {
    return movimentoCollection;
}

public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
    this.movimentoCollection = movimentoCollection;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idPessoa != null ? idPessoa.hashCode() : 0);
    return hash;
}
```

```
    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Pessoa)) {
            return false;
        }
        Pessoa other = (Pessoa) object;
        return !((this.idPessoa == null && other.idPessoa != null)
                || (this.idPessoa != null && !this.idPessoa.equals(other.idPessoa)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.Pessoa[ idPessoa=" + idPessoa + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/model/PessoaFisica.java

```
package cadastroee.model;

import java.io.Serializable;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

/**
 * Entidade JPA para a tabela PessoaFisica (gerada a partir do banco "loja").
 */
@Entity
@Table(name = "PessoaFisica")
@NamedQueries({
    @NamedQuery(name = "PessoaFisica.findAll", query = "SELECT p FROM PessoaFisica p"),
    @NamedQuery(name = "PessoaFisica.findByIdPessoa", query = "SELECT p FROM PessoaFisica p WHERE
p.idPessoa = :idPessoa"),
    @NamedQuery(name = "PessoaFisica.findByCpf", query = "SELECT p FROM PessoaFisica p WHERE p.cpf =
:cpf")})
public class PessoaFisica implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @Basic(optional = false)
    @NotNull
    @Column(name = "idPessoa")
    private Integer idPessoa;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 11)
    @Column(name = "cpf")
    private String cpf;
    @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa", insertable = false, updatable
= false)
```

```
    @OneToOne(optional = false)
    private Pessoa pessoa;

    public PessoaFisica() {
    }

    public PessoaFisica(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }

    public Integer getIdPessoa() {
        return idPessoa;
    }

    public void setIdPessoa(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public Pessoa getPessoa() {
        return pessoa;
    }

    public void setPessoa(Pessoa pessoa) {
        this.pessoa = pessoa;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (idPessoa != null ? idPessoa.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        if (!(object instanceof PessoaFisica)) {
            return false;
        }
        PessoaFisica other = (PessoaFisica) object;
        return !((this.idPessoa == null && other.idPessoa != null)
                || (this.idPessoa != null && !this.idPessoa.equals(other.idPessoa)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.PessoaFisica[ idPessoa=" + idPessoa + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/model/PessoaJuridica.java

```
package cadastroee.model;

import java.io.Serializable;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

/**
 * Entidade JPA para a tabela PessoaJuridica (gerada a partir do banco "loja").
 */
@Entity
@Table(name = "PessoaJuridica")
@NamedQueries({
    @NamedQuery(name = "PessoaJuridica.findAll", query = "SELECT p FROM PessoaJuridica p"),
    @NamedQuery(name = "PessoaJuridica.findByIdPessoa", query = "SELECT p FROM PessoaJuridica p
WHERE p.idPessoa = :idPessoa"),
    @NamedQuery(name = "PessoaJuridica.findByCnpj", query = "SELECT p FROM PessoaJuridica p WHERE
p.cnpj = :cnpj")})
public class PessoaJuridica implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @Basic(optional = false)
    @NotNull
    @Column(name = "idPessoa")
    private Integer idPessoa;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 14)
    @Column(name = "cnpj")
    private String cnpj;
    @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa", insertable = false, updatable
= false)
    @OneToOne(optional = false)
    private Pessoa pessoa;

    public PessoaJuridica() {
    }

    public PessoaJuridica(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }

    public Integer getIdPessoa() {
        return idPessoa;
    }

    public void setIdPessoa(Integer idPessoa) {
        this.idPessoa = idPessoa;
    }
```

```
    public String getCnpj() {
        return cnpj;
    }

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }

    public Pessoa getPessoa() {
        return pessoa;
    }

    public void setPessoa(Pessoa pessoa) {
        this.pessoa = pessoa;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (idPessoa != null ? idPessoa.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        if (!(object instanceof PessoaJuridica)) {
            return false;
        }
        PessoaJuridica other = (PessoaJuridica) object;
        return !((this.idPessoa == null && other.idPessoa != null)
                || (this.idPessoa != null && !this.idPessoa.equals(other.idPessoa)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.PessoaJuridica[ idPessoa=" + idPessoa + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/model/Usuario.java

```
package cadastroee.model;

import java.io.Serializable;
import java.util.Collection;
import jakarta.persistence.Basic;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
```

```
/**
 * Entidade JPA para a tabela Usuario (gerada a partir do banco "loja").
 */
@Entity
@Table(name = "Usuario")
@NamedQueries({
    @NamedQuery(name = "Usuario.findAll", query = "SELECT u FROM Usuario u"),
    @NamedQuery(name = "Usuario.findByIdUsuario", query = "SELECT u FROM Usuario u WHERE u.idUsuario
= :idUsuario"),
    @NamedQuery(name = "Usuario.findByLogin", query = "SELECT u FROM Usuario u WHERE u.login
= :login"),
    @NamedQuery(name = "Usuario.findBySenha", query = "SELECT u FROM Usuario u WHERE u.senha
= :senha")})
public class Usuario implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "idUsuario")
    private Integer idUsuario;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 50)
    @Column(name = "login")
    private String login;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 50)
    @Column(name = "senha")
    private String senha;
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "idUsuario")
    private Collection<Movimento> movimentoCollection;

    public Usuario() {
    }

    public Usuario(Integer idUsuario) {
        this.idUsuario = idUsuario;
    }

    public Integer getIdUsuario() {
        return idUsuario;
    }

    public void setIdUsuario(Integer idUsuario) {
        this.idUsuario = idUsuario;
    }

    public String getLogin() {
        return login;
    }

    public void setLogin(String login) {
        this.login = login;
    }

    public String getSenha() {
        return senha;
    }
```

```
    public void setSenha(String senha) {
        this.senha = senha;
    }

    public Collection<Movimento> getMovimentoCollection() {
        return movimentoCollection;
    }

    public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
        this.movimentoCollection = movimentoCollection;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (idUsuario != null ? idUsuario.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Usuario)) {
            return false;
        }
        Usuario other = (Usuario) object;
        return !((this.idUsuario == null && other.idUsuario != null)
                || (this.idUsuario != null && !this.idUsuario.equals(other.idUsuario)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.Usuario[ idUsuario=" + idUsuario + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/model/Movimento.java

```
package cadastroee.model;

import java.io.Serializable;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

/**
 * Entidade JPA para a tabela Movimento (gerada a partir do banco "loja").
 */
@Entity
```

```
@Table(name = "Movimento")
@NamedQueries({
    @NamedQuery(name = "Movimento.findAll", query = "SELECT m FROM Movimento m"),
    @NamedQuery(name = "Movimento.findByIdMovimento", query = "SELECT m FROM Movimento m WHERE
m.idMovimento = :idMovimento"),
    @NamedQuery(name = "Movimento.findByQuantidade", query = "SELECT m FROM Movimento m WHERE
m.quantidade = :quantidade"),
    @NamedQuery(name = "Movimento.findByTipo", query = "SELECT m FROM Movimento m WHERE m.tipo
= :tipo"),
    @NamedQuery(name = "Movimento.findByValorUnitario", query = "SELECT m FROM Movimento m WHERE
m.valorUnitario = :valorUnitario")})
public class Movimento implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "idMovimento")
    private Integer idMovimento;
    @Basic(optional = false)
    @NotNull
    @Column(name = "quantidade")
    private int quantidade;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 1)
    @Column(name = "tipo")
    private String tipo;
    @Basic(optional = false)
    @NotNull
    @Column(name = "valorUnitario")
    private Float valorUnitario;
    @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa")
    @ManyToOne(optional = false)
    private Pessoa idPessoa;
    @JoinColumn(name = "idProduto", referencedColumnName = "idProduto")
    @ManyToOne(optional = false)
    private Produto idProduto;
    @JoinColumn(name = "idUsuario", referencedColumnName = "idUsuario")
    @ManyToOne(optional = false)
    private Usuario idUsuario;

    public Movimento() {
    }

    public Movimento(Integer idMovimento) {
        this.idMovimento = idMovimento;
    }

    public Integer getIdMovimento() {
        return idMovimento;
    }

    public void setIdMovimento(Integer idMovimento) {
        this.idMovimento = idMovimento;
    }

    public int getQuantidade() {
        return quantidade;
    }
```

```
public void setQuantidade(int quantidade) {
    this.quantidade = quantidade;
}

public String getTipo() {
    return tipo;
}

public void setTipo(String tipo) {
    this.tipo = tipo;
}

public Float getValorUnitario() {
    return valorUnitario;
}

public void setValorUnitario(Float valorUnitario) {
    this.valorUnitario = valorUnitario;
}

public Pessoa getIdPessoa() {
    return idPessoa;
}

public void setIdPessoa(Pessoa idPessoa) {
    this.idPessoa = idPessoa;
}

public Produto getIdProduto() {
    return idProduto;
}

public void setIdProduto(Produto idProduto) {
    this.idProduto = idProduto;
}

public Usuario getIdUsuario() {
    return idUsuario;
}

public void setIdUsuario(Usuario idUsuario) {
    this.idUsuario = idUsuario;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idMovimento != null ? idMovimento.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    if (!(object instanceof Movimento)) {
        return false;
    }
    Movimento other = (Movimento) object;
    return !((this.idMovimento == null && other.idMovimento != null)
            || (this.idMovimento != null && !this.idMovimento.equals(other.idMovimento)));
```

```
    }

    @Override
    public String toString() {
        return "cadastroee.model.Movimento[ idMovimento=" + idMovimento + " ]";
    }

}
```

CadastroEE-ejb/src/java/cadastroee/controller/AbstractFacade.java

```
package cadastroee.controller;

import java.util.List;
import jakarta.persistence.EntityManager;
import jakarta.persistence.criteria.CriteriaQuery;
import jakarta.persistence.criteria.Root;

/**
 * Classe base dos Session Beans (gerada por "New Session Beans for Entity Classes").
 * Implementa as operações genéricas de persistência via EntityManager.
 */
public abstract class AbstractFacade<T> {

    private final Class<T> entityClass;

    public AbstractFacade(Class<T> entityClass) {
        this.entityClass = entityClass;
    }

    protected abstract EntityManager getEntityManager();

    public void create(T entity) {
        getEntityManager().persist(entity);
    }

    public void edit(T entity) {
        getEntityManager().merge(entity);
    }

    public void remove(T entity) {
        getEntityManager().remove(getEntityManager().merge(entity));
    }

    public T find(Object id) {
        return getEntityManager().find(entityClass, id);
    }

    public List<T> findAll() {
        CriteriaQuery<T> cq = getEntityManager().getCriteriaBuilder().createQuery(entityClass);
        cq.select(cq.from(entityClass));
        return getEntityManager().createQuery(cq).getResultList();
    }

    public List<T> findRange(int[] range) {
        CriteriaQuery<T> cq = getEntityManager().getCriteriaBuilder().createQuery(entityClass);
        cq.select(cq.from(entityClass));
        jakarta.persistence.Query q = getEntityManager().createQuery(cq);
        q.setMaxResults(range[1] - range[0] + 1);
        q.setFirstResult(range[0]);
```

```
        return q.getResultList();
    }

    public int count() {
        CriteriaQuery<Long> cq = getEntityManager().getCriteriaBuilder().createQuery(Long.class);
        Root<T> rt = cq.from(entityClass);
        cq.select(getEntityManager().getCriteriaBuilder().count(rt));
        jakarta.persistence.Query q = getEntityManager().createQuery(cq);
        return ((Long) q.getSingleResult()).intValue();
    }

}
```

CadastroEE-ejb/src/java/cadastroee/controller/ProdutoFacade.java

```
package cadastroee.controller;

import cadastroee.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

/**
 * Session Bean (EJB stateless) para a entidade Produto.
 */
@Stateless
public class ProdutoFacade extends AbstractFacade<Produto> implements ProdutoFacadeLocal {

    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        return em;
    }

    public ProdutoFacade() {
        super(Produto.class);
    }

}
```

CadastroEE-ejb/src/java/cadastroee/controller/ProdutoFacadeLocal.java

```
package cadastroee.controller;

import cadastroee.model.Produto;
import java.util.List;
import jakarta.ejb.Local;

/**
 * Interface local do Session Bean ProdutoFacade.
 */
@Local
public interface ProdutoFacadeLocal {

    void create(Produto produto);

    void edit(Produto produto);
```

```
    void remove(Produto produto);

    Produto find(Object id);

    List<Produto> findAll();

    List<Produto> findRange(int[] range);

    int count();

}
```

As demais classes MovimentoFacade, PessoaFacade, PessoaFisicaFacade, PessoaJuridicaFacade e UsuarioFacade (e suas interfaces ...FacadeLocal) seguem exatamente o mesmo modelo, mudando apenas a entidade, e estão no repositório.

1.5 Códigos – CadastroEE-war (Servlet de teste)

CadastroEE-war/web/WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-
app_4_0.xsd">
    <servlet>
        <servlet-name>ServletProduto</servlet-name>
        <servlet-class>cadastroee.servlets.ServletProduto</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>ServletProdutoFC</servlet-name>
        <servlet-class>cadastroee.servlets.ServletProdutoFC</servlet-class>
    </servlet>
    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
</web-app>
```

CadastroEE-war/src/java/cadastroee/servlets/ServletProduto.java

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * Servlet de teste: recupera os produtos através do EJB (ProdutoFacadeLocal)
 * e os apresenta como uma lista HTML.
```

```
 */
@WebServlet(name = "ServletProduto", urlPatterns = {"/ServletProduto"})
public class ServletProduto extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        List<Produto> produtos = facade.findAll();
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<meta charset=\"UTF-8\">");
            out.println("<title>Servlet ServletProduto</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet ServletProduto at " + request.getContextPath() + "</h1>");
            out.println("<h2>Produtos cadastrados</h2>");
            out.println("<ul>");
            for (Produto produto : produtos) {
                out.println("<li>" + Html.esc(produto.getNome()) + "</li>");
            }
            out.println("</ul>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Lista os produtos cadastrados via EJB";
    }

}
```

CadastroEE-war/src/java/cadastroee/servlets/Html.java

```
package cadastroee.servlets;

/**
 * Utilitário para escapar texto antes de escrevê-lo em HTML
 * (evita que um nome de produto com "&lt;" quebre a página ou injete script).
 */
public final class Html {
```

```
    private Html() {
    }

    public static String esc(Object valor) {
        if (valor == null) {
            return "";
        }
        String s = valor.toString();
        StringBuilder sb = new StringBuilder(s.length());
        for (char c : s.toCharArray()) {
            switch (c) {
                case '<': sb.append("&lt;"); break;
                case '>': sb.append("&gt;"); break;
                case '&': sb.append("&amp;"); break;
                case '"': sb.append("&quot;"); break;
                case '\'': sb.append("&#39;"); break;
                default: sb.append(c);
            }
        }
        return sb.toString();
    }
}
```

1.6 Execução

O projeto principal CadastroEE foi executado com Run, que compilou os três módulos, gerou o EAR e o implantou no GlassFish. Com o banco alimentado pelo script de dados, o ServletProduto apresentou a lista de produtos obtida pelo EJB:

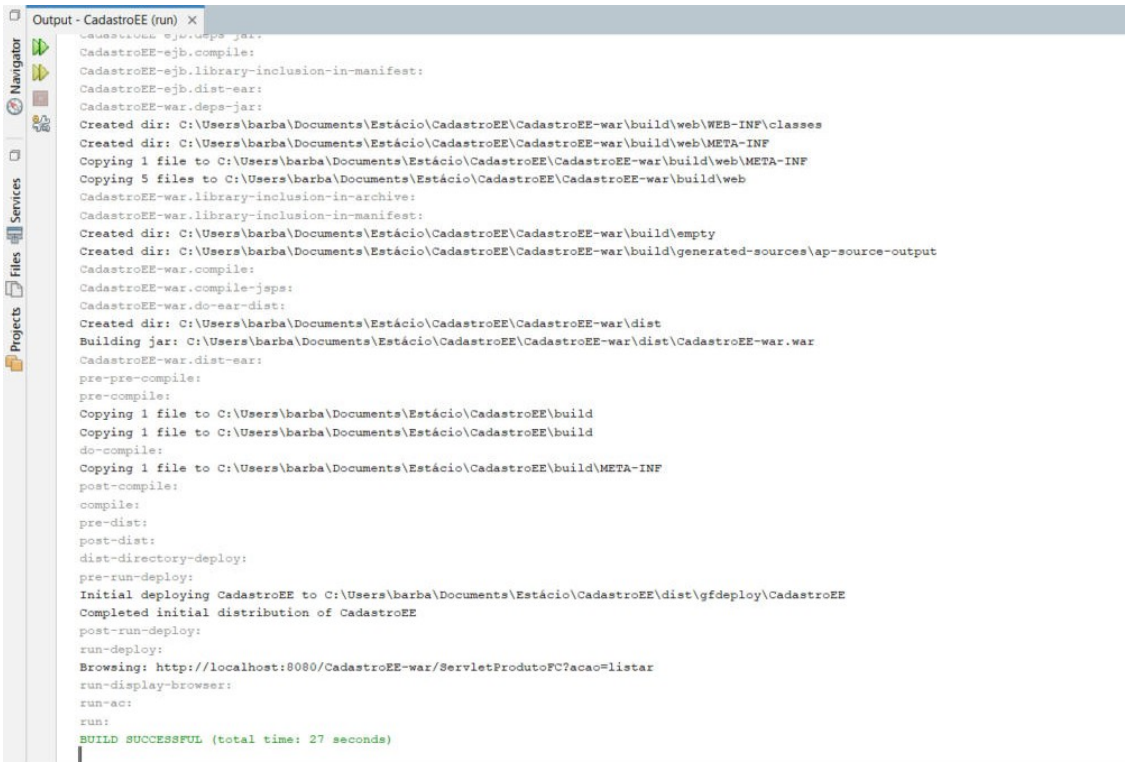


Figura 3 – Saída do Run no NetBeans: build dos três módulos, implantação no GlassFish e BUILD SUCCESSFUL

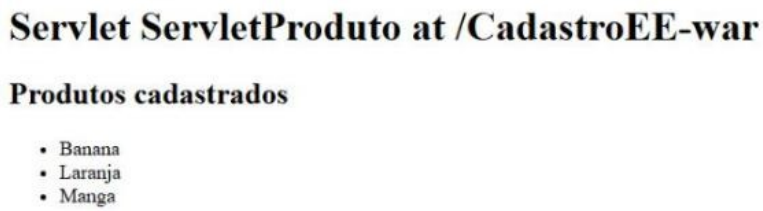


Figura 4 – http://localhost:8080/CadastroEE-war/ServletProduto

1.7 Análise e Conclusão

Como é organizado um projeto corporativo no NetBeans?

O NetBeans organiza um aplicativo corporativo (Enterprise Application, projeto Ant) em três projetos relacionados. O projeto principal, CadastroEE, não tem código próprio: ele apenas agrega os módulos e gera o arquivo CadastroEE.ear, que é a unidade implantada no GlassFish. O CadastroEE-ejb é o módulo EJB, onde ficam a camada de persistência (entidades JPA no pacote cadastroee.model e o persistence.xml) e a camada de controle (Session Beans e interfaces locais no pacote cadastroee.controller); ele gera um .jar. O CadastroEE-war é o módulo Web, com Servlets, JSPs, recursos estáticos e o web.xml; ele gera um .war e depende do módulo EJB para enxergar as interfaces dos Session Beans. Cada projeto tem seu build.xml e a pasta nbproject com as configurações (servidor de destino, bibliotecas, plataforma Jakarta EE), e o NetBeans resolve as dependências entre eles ao compilar e implantar o projeto principal.

Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?

A JPA (Jakarta Persistence) faz o mapeamento objeto-relacional: cada tabela do banco loja vira uma classe anotada com @Entity, e o EntityManager traduz operações sobre objetos (persist, merge, remove, find, consultas JPQL) em comandos SQL, sem que a aplicação precise escrever JDBC manualmente. O provedor (EclipseLink, no GlassFish) usa o DataSource jdbc/loja, obtido por JNDI a partir do pool SQLServerPool. Os EJBs (Enterprise JavaBeans) concentram as regras de negócio em componentes gerenciados pelo servidor: o container cria e reaproveita as instâncias em um pool, injeta o EntityManager (@PersistenceContext), controla as transações automaticamente (cada método de um @Stateless roda em uma transação JTA) e cuida de segurança e concorrência. Assim, a camada Web apenas chama métodos de negócio, e a persistência e as transações ficam a cargo do servidor de aplicação.

Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?

Por meio de assistentes que geram código a partir do banco de dados. O "New Entity Classes from Database" lê as tabelas pelo Data Source jdbc/loja e cria todas as entidades com @Entity, @Table, @Id, @Column, relacionamentos (@OneToMany, @ManyToOne, @OneToOne), validações (@NotNull, @Size) e consultas nomeadas, além do persistence.xml. Em seguida, o "New Session Beans for Entity Classes" gera um Session Bean @Stateless (sufixo Facade) e uma interface @Local (sufixo FacadeLocal) para cada entidade, todos herdando de AbstractFacade com o CRUD completo (create, edit, remove, find, findAll, findRange e count). Somam-se a isso a integração com o GlassFish (iniciar, implantar e ver logs pela aba Serviços), a aba de Banco de Dados para consultar o SQL Server, o autocompletar de anotações e a implantação com um clique em Run.

O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes em um projeto Web?

Servlets são classes Java que estendem HttpServlet e rodam no container Web do servidor, respondendo a requisições HTTP: o container chama doGet ou doPost, entregando os objetos HttpServletRequest (dados da requisição) e HttpServletResponse (resposta a ser gerada). São o ponto de entrada da camada Web e podem gerar HTML diretamente ou encaminhar a requisição para uma JSP. No NetBeans, o assistente New › Servlet

gera a classe com o método processRequest e os métodos doGet/doPost já ligados a ele, permite definir nome, pacote e padrão de URL e, com a opção "Add information to deployment descriptor", registra o Servlet no web.xml. O editor ainda oferece a inserção de chamadas a EJBs (Insert Code › Call Enterprise Bean), que adiciona o atributo anotado com @EJB.

Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

Por injeção de dependência. No Servlet é declarado um atributo do tipo da interface local, anotado com @EJB (ProdutoFacadeLocal facade). Ao instanciar o Servlet, o container localiza o Session Bean via JNDI e injeta uma referência (um proxy) para ele. Quando o Servlet chama facade.findAll(), o container obtém uma instância livre do pool de ProdutoFacade (que é @Stateless), abre a transação, executa o método e devolve a instância ao pool. Como os módulos estão no mesmo EAR e na mesma JVM, a chamada usa a interface @Local, com passagem de parâmetros por referência e sem custo de comunicação remota.

2º Procedimento – Interface Cadastral com Servlet e JSPs

Foi criado o Servlet ServletProdutoFC no padrão Front Controller: ele captura o parâmetro acao, executa a operação correspondente pelo ProdutoFacadeLocal e encaminha (forward via RequestDispatcher) para ProdutoDados.jsp (formIncluir e formAlterar) ou ProdutoLista.jsp (demais ações). Também foram criadas as páginas de consulta e de cadastro. O tratamento de erro (try/catch) mostra uma mensagem na listagem quando uma operação falha, por exemplo ao excluir um produto que possui movimentos (chave estrangeira).

2.1 Códigos

CadastroEE-war/src/java/cadastroee/servlets/ServletProdutoFC.java

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import java.io.IOException;
import jakarta.ejb.EJB;
import jakarta.ejb.EJBException;
import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * Front Controller do cadastro de produtos.
 *
 * Toda requisição passa por este Servlet, que lê o parâmetro "acao",
 * executa a regra de negócio através do EJB (ProdutoFacadeLocal) e
 * encaminha (forward) para a JSP de apresentação adequada:
 *   - formIncluir / formAlterar -> ProdutoDados.jsp
 *   - listar / incluir / alterar / excluir -> ProdutoLista.jsp
 */
@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");

        String acao = request.getParameter("acao");
        if (acao == null || acao.isEmpty()) {
            acao = "listar";
        }

        String destino = ("formAlterar".equals(acao) || "formIncluir".equals(acao))
                ? "ProdutoDados.jsp"
                : "ProdutoLista.jsp";

        try {
            switch (acao) {
                case "listar":
                    request.setAttribute("lista", facade.findAll());
                    break;
```

```
            case "formAlterar": {
                Integer id = Integer.valueOf(request.getParameter("id"));
                request.setAttribute("produto", facade.find(id));
                break;
            }

            case "formIncluir":
                // formulário vazio: nenhuma entidade é enviada
                break;

            case "excluir": {
                Integer id = Integer.valueOf(request.getParameter("id"));
                Produto produto = facade.find(id);
                if (produto != null) {
                    facade.remove(produto);
                }
                request.setAttribute("lista", facade.findAll());
                break;
            }

            case "alterar": {
                Integer id = Integer.valueOf(request.getParameter("id"));
                Produto produto = facade.find(id);
                preencher(produto, request);
                facade.edit(produto);
                request.setAttribute("lista", facade.findAll());
                break;
            }

            case "incluir": {
                Produto produto = new Produto();
                preencher(produto, request);
                facade.create(produto);
                request.setAttribute("lista", facade.findAll());
                break;
            }

            default:
                request.setAttribute("lista", facade.findAll());
        }
    } catch (EJBException e) {
        // ex.: excluir um produto que possui movimentos (chave estrangeira)
        request.setAttribute("erro", "Não foi possível concluir a operação \"" + acao
                + "\". Verifique se o produto não possui movimentos associados.");
        request.setAttribute("lista", facade.findAll());
        destino = "ProdutoLista.jsp";
    } catch (NumberFormatException e) {
        request.setAttribute("erro", "Valor numérico inválido: " + e.getMessage());
        request.setAttribute("lista", facade.findAll());
        destino = "ProdutoLista.jsp";
    }

    RequestDispatcher rd = request.getRequestDispatcher(destino);
    rd.forward(request, response);
}

/** Copia nome, quantidade e preço de venda do request para a entidade. */
private void preencher(Produto produto, HttpServletRequest request) {
    produto.setNome(request.getParameter("nome").trim());
```

```
        produto.setQuantidade(Integer.parseInt(request.getParameter("quantidade").trim()));
        produto.setPrecoVenda(Float.valueOf(request.getParameter("precoVenda").trim().replace(",",
".")));
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Front Controller do cadastro de produtos";
    }

}
```

ProdutoLista.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="cadastroee.model.Produto"%>
<%@page import="cadastroee.servlets.Html"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Listagem de Produtos</title>
    </head>
    <body>
        <h1>Listagem de Produtos</h1>
        <a href="ServletProdutoFC?acao=formIncluir">Novo Produto</a>
        <% String erro = (String) request.getAttribute("erro");
           if (erro != null) { %>
        <p style="color: red;"><%= Html.esc(erro) %></p>
        <% } %>
        <table border="1">
            <thead>
                <tr>
                    <th>#</th>
                    <th>Nome</th>
                    <th>Quantidade</th>
                    <th>Preço de Venda</th>
                    <th>Opções</th>
                </tr>
            </thead>
            <tbody>
                <%
                    List<Produto> lista = (List<Produto>) request.getAttribute("lista");
                    if (lista != null) {
                        for (Produto produto : lista) {
                %>
                <tr>
```

```
                    <td><%= produto.getIdProduto() %></td>
                    <td><%= Html.esc(produto.getNome()) %></td>
                    <td><%= produto.getQuantidade() %></td>
                    <td><%= String.format("%.2f", produto.getPrecoVenda()) %></td>
                    <td>
                        <a href="ServletProdutoFC?acao=formAlterar&id=<%= produto.getIdProduto()
%>">Alterar</a>
                        <a href="ServletProdutoFC?acao=excluir&id=<%= produto.getIdProduto()
%>">Excluir</a>
                    </td>
                </tr>
                <%
                        }
                    }
                %>
            </tbody>
        </table>
    </body>
</html>
```

ProdutoDados.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>
<%@page import="cadastroee.servlets.Html"%>
<%
    Produto produto = (Produto) request.getAttribute("produto");
    String acao = (produto == null) ? "incluir" : "alterar";
%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Dados do Produto</title>
    </head>
    <body>
        <h1><%= produto == null ? "Incluir Produto" : "Alterar Produto" %></h1>
        <form action="ServletProdutoFC" method="post">
            <input type="hidden" name="acao" value="<%= acao %>">
            <% if ("alterar".equals(acao)) { %>
            <input type="hidden" name="id" value="<%= produto.getIdProduto() %>">
            <% } %>
            <p>
                <label for="nome">Nome:</label>
                <input type="text" id="nome" name="nome" required
                       value="<%= produto == null ? "" : Html.esc(produto.getNome()) %>">
            </p>
            <p>
                <label for="quantidade">Quantidade:</label>
                <input type="number" id="quantidade" name="quantidade" min="0" required
                       value="<%= produto == null ? "" : produto.getQuantidade() %>">
            </p>
            <p>
                <label for="precoVenda">Preço de Venda:</label>
                <input type="number" id="precoVenda" name="precoVenda" min="0" step="0.01" required
                       value="<%= produto == null ? "" : String.format(java.util.Locale.US, "%.2f",
produto.getPrecoVenda()) %>">
            </p>
            <input type="submit" value="<%= produto == null ? "Incluir" : "Alterar" %>">
        </form>
```

```
    </body>
</html>
```

2.2 Resultados dos testes

Listagem

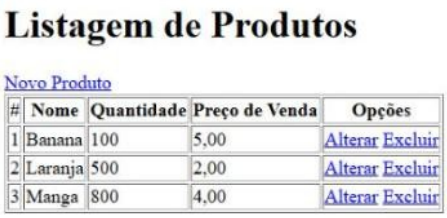


Figura 5 – http://localhost:8080/CadastroEE-war/ServletProdutoFC?acao=listar

Inclusão (a partir do link "Novo Produto")

Foi incluído o produto "Maçã" (para testar a acentuação, tratada com request.setCharacterEncoding("UTF-8")):

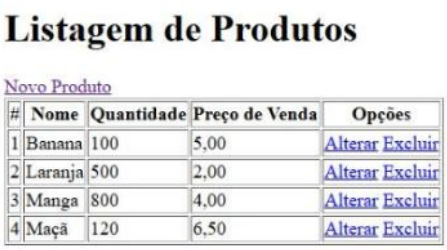


Figura 6 – Listagem após a inclusão do produto Maçã

Alteração (a partir do link dinâmico "Alterar")

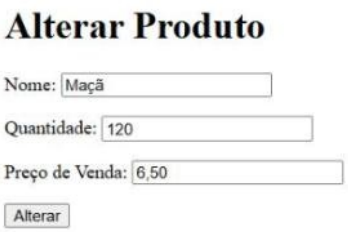


Figura 7 – Formulário de alteração aberto com os dados do produto 4

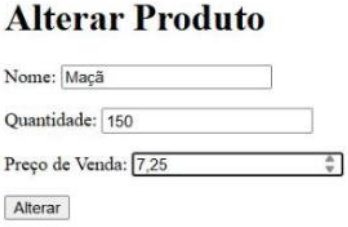


Figura 8 – Quantidade alterada para 150 e preço para 7,25

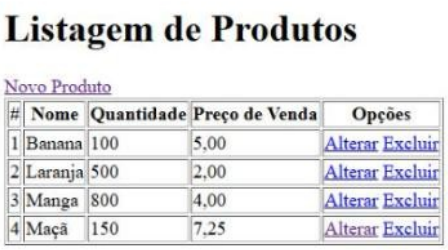


Figura 9 – Listagem após a alteração

Exclusão (a partir do link dinâmico "Excluir")

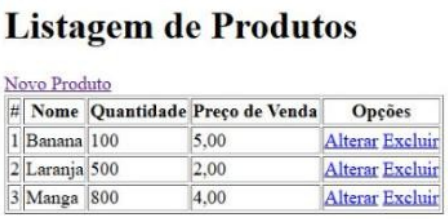


Figura 10 – Listagem após excluir o produto Maçã



Figura 11 – Tentativa de excluir Banana, que possui movimentos: a operação é recusada e a mensagem é exibida

Nova inclusão



Figura 12 – Formulário de inclusão (acao=formIncluir)

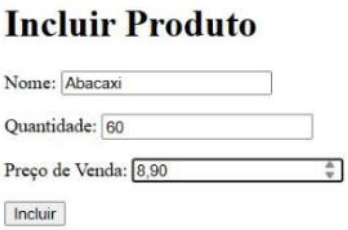


Figura 13 – Formulário preenchido com o produto Abacaxi



Figura 14 – Listagem após a inclusão do Abacaxi

2.3 Análise e Conclusão

Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

No Front Controller, um único componente recebe todas as requisições de uma funcionalidade e decide o que fazer com cada uma, em vez de cada página ter seu próprio controlador. Em Java Web ele é implementado por um Servlet, aqui o ServletProdutoFC: ele lê o parâmetro acao (listar, incluir, alterar, excluir, formIncluir, formAlterar), aciona o Model (o EJB ProdutoFacadeLocal e a entidade Produto) para executar a regra de negócio, coloca os dados necessários como atributos do request e escolhe a View (ProdutoLista.jsp ou ProdutoDados.jsp), para a qual faz o forward. Assim, no MVC o Servlet é o Controller, as JSPs são as Views e os EJBs com as entidades JPA formam o Model, o que centraliza o controle de fluxo, facilita tratar erros e evita lógica de negócio dentro das páginas.

Quais as diferenças e semelhanças entre Servlets e JSPs?

Semelhanças: ambos rodam no container Web, têm acesso aos mesmos objetos (request, response, session, application) e geram respostas HTTP dinâmicas. Na verdade, uma JSP é traduzida pelo servidor em um Servlet e compilada no primeiro acesso. Diferenças: o Servlet é uma classe Java em que o HTML precisa ser escrito dentro do código (out.println), o que o torna adequado para controle, processamento e regras; a JSP é um documento HTML com trechos Java (scriptlets <% %>, expressões <%= %>, diretivas <%@ %>) ou tags, sendo mais natural para a apresentação. Por isso, no MVC o Servlet faz o papel de controlador e a JSP o de visão.

Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

O redirecionamento simples (response.sendRedirect) devolve ao navegador uma resposta 302 com uma nova URL; o navegador faz uma segunda requisição, a barra de endereços muda e os dados do request original se perdem. O forward (request.getRequestDispatcher(destino).forward(request, response)) acontece dentro do servidor: a mesma requisição é repassada para outro recurso (a JSP), em uma única viagem, sem que o navegador saiba, preservando os atributos do request. Parâmetros são os dados enviados pelo cliente (query string ou campos do formulário), sempre do tipo String e somente leitura, lidos com getParameter (ex.: acao, id, nome). Atributos são objetos Java que o servidor coloca no request com setAttribute e recupera com getAttribute, usados para passar dados entre componentes no servidor, como a lista de produtos ou o produto em edição que o Servlet entrega às JSPs.

3º Procedimento – Melhorando o Design da Interface

As bibliotecas do Bootstrap foram incluídas por CDN (link do CSS e script do JavaScript copiados de getbootstrap.com) na divisão head de ProdutoLista.jsp e ProdutoDados.jsp, e foram aplicadas as classes pedidas: container no body; btn btn-primary m-2 no link de inclusão; table table-striped na tabela e table-dark no thead; btn btn-primary btn-sm e btn btn-danger btn-sm nos links de alteração e exclusão; form no formulário, div mb-3 envolvendo cada par label/input, form-label nos labels, form-control nos inputs e btn btn-primary no botão.

3.1 Códigos

CadastroEE-war/web/ProdutoLista.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="cadastroee.model.Produto"%>
<%@page import="cadastroee.servlets.Html"%>
<!DOCTYPE html>
<html lang="pt-BR">
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Listagem de Produtos</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
    </head>
    <body class="container">
        <h1 class="mt-3">Listagem de Produtos</h1>
        <a class="btn btn-primary m-2" href="ServletProdutoFC?acao=formIncluir">Novo Produto</a>
        <% String erro = (String) request.getAttribute("erro");
           if (erro != null) { %>
        <div class="alert alert-danger" role="alert"><%= Html.esc(erro) %></div>
        <% } %>
        <table class="table table-striped">
            <thead class="table-dark">
                <tr>
                    <th>#</th>
                    <th>Nome</th>
                    <th>Quantidade</th>
                    <th>Preço de Venda</th>
                    <th>Opções</th>
                </tr>
            </thead>
            <tbody>
                <%
                    List<Produto> lista = (List<Produto>) request.getAttribute("lista");
                    if (lista != null) {
                        for (Produto produto : lista) {
                %>
                <tr>
                    <td><%= produto.getIdProduto() %></td>
                    <td><%= Html.esc(produto.getNome()) %></td>
                    <td><%= produto.getQuantidade() %></td>
                    <td><%= String.format("%.2f", produto.getPrecoVenda()) %></td>
                    <td>
```

```
                        <a class="btn btn-primary btn-sm" href="ServletProdutoFC?
acao=formAlterar&id=<%= produto.getIdProduto() %>">Alterar</a>
                        <a class="btn btn-danger btn-sm" href="ServletProdutoFC?acao=excluir&id=<%=
produto.getIdProduto() %>">Excluir</a>
                    </td>
                </tr>
                <%
                        }
                    }
                %>
            </tbody>
        </table>
    </body>
</html>
```

CadastroEE-war/web/ProdutoDados.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>
<%@page import="cadastroee.servlets.Html"%>
<%
    Produto produto = (Produto) request.getAttribute("produto");
    String acao = (produto == null) ? "incluir" : "alterar";
%>
<!DOCTYPE html>
<html lang="pt-BR">
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Dados do Produto</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
    </head>
    <body class="container">
        <h1 class="mt-3"><%= produto == null ? "Incluir Produto" : "Alterar Produto" %></h1>
        <form class="form" action="ServletProdutoFC" method="post">
            <input type="hidden" name="acao" value="<%= acao %>">
            <% if ("alterar".equals(acao)) { %>
            <input type="hidden" name="id" value="<%= produto.getIdProduto() %>">
            <% } %>
            <div class="mb-3">
                <label class="form-label" for="nome">Nome</label>
                <input class="form-control" type="text" id="nome" name="nome" required
                       value="<%= produto == null ? "" : Html.esc(produto.getNome()) %>">
            </div>
            <div class="mb-3">
                <label class="form-label" for="quantidade">Quantidade</label>
                <input class="form-control" type="number" id="quantidade" name="quantidade" min="0"
required
                       value="<%= produto == null ? "" : produto.getQuantidade() %>">
            </div>
            <div class="mb-3">
                <label class="form-label" for="precoVenda">Preço de Venda</label>
                <input class="form-control" type="number" id="precoVenda" name="precoVenda" min="0"
step="0.01" required
```

```
                        value="<%= produto == null ? "" : String.format(java.util.Locale.US, "%.2f",
produto.getPrecoVenda()) %>">
            </div>
            <input class="btn btn-primary" type="submit" value="<%= produto == null ? "Incluir" :
"Alterar" %>">
        </form>
    </body>
</html>
```

3.2 Resultados



Figura 15 – Listagem com Bootstrap



Figura 16 – Formulário de inclusão com Bootstrap



Figura 17 – Inclusão do produto Uva



Figura 18 – Listagem após a inclusão



Figura 19 – Alteração do produto Uva (quantidade 180, preço 10,50)



Figura 20 – Listagem após a alteração



Figura 21 – Listagem após excluir o Abacaxi



Figura 22 – Mensagem de erro com o componente alert do Bootstrap

3.3 Análise e Conclusão

Como o framework Bootstrap é utilizado?

O Bootstrap é incluído nas páginas por CDN: um <link> para o arquivo bootstrap.min.css e um <script> para o bootstrap.bundle.min.js, no <head> de ProdutoLista.jsp e ProdutoDados.jsp. A partir daí, basta aplicar as classes prontas aos elementos HTML: container no body, btn btn-primary m-2 no link de inclusão, table table-striped na tabela e table-dark no cabeçalho, btn btn-primary btn-sm e btn btn-danger btn-sm nos links de

alterar e excluir, e no formulário as classes form, mb-3 (agrupando cada label/input), form-label, form-control e btn btn-primary no botão. Nenhum CSS próprio precisou ser escrito.

Por que o Bootstrap garante a independência estrutural do HTML?

Porque toda a aparência é definida por classes CSS do framework, e não pela estrutura ou por atributos visuais no HTML. A marcação continua semântica (table, thead, form, label, input, a) e o visual muda apenas acrescentando ou trocando classes. O mesmo documento pode ser exibido de forma totalmente diferente trocando o tema ou removendo o CSS, sem alterar a lógica das JSPs. Isso separa conteúdo de apresentação e permite evoluir o design sem mexer no código Java.

Qual a relação entre o Bootstrap e a responsividade da página?

O Bootstrap é construído com abordagem mobile-first: usa media queries e breakpoints (sm, md, lg, xl, xxl), o sistema de grid em 12 colunas e componentes flexíveis, de forma que o layout se adapta à largura da tela. A classe container ajusta a largura máxima do conteúdo a cada breakpoint, os campos form-control ocupam 100% da largura disponível e as tabelas e botões se reorganizam em telas pequenas. Junto com a meta tag viewport (width=device-width, initial-scale=1), as páginas ficam legíveis em celulares, tablets e desktops sem CSS adicional.

Conclusão geral

O trabalho percorreu as camadas de uma aplicação corporativa Java: as entidades JPA mapeiam o banco loja, os Session Beans concentram as operações e as transações, o Servlet Front Controller organiza o fluxo da interface e as JSPs, estilizadas com Bootstrap, apresentam os dados. Os assistentes do NetBeans geraram boa parte do código repetitivo, e os ajustes para o GlassFish 6 (troca de javax por jakarta, chave IDENTITY e tipo Float) mostraram na prática como versões de plataforma e mapeamento objeto-relacional influenciam o projeto.